

POINTERS

1. What Is a Pointer?

A **pointer** is a variable that **stores the memory address** of another variable.

```
int x = 10;
```

```
int* p = &x; // p stores the address of x
```

- $x \rightarrow$ value 10
 - $\&x \rightarrow$ address of x
 - $p \rightarrow$ holds address of x
 - $*p \rightarrow$ value at that address (10)
-

Memory Picture

$x = 10$ (stored at address 0x100)

$p = 0x100$

$*p = 10$

2. Declaring and Using Pointers

```
int a = 5;
```

```
int* ptr = &a;
```

```
cout << a << endl; // 5
```

```
cout << ptr << endl; // address of a
```

```
cout << *ptr << endl; // 5
```

Key Symbols

Symbol Meaning

$\&$ Address-of operator

Symbol Meaning

* Dereference operator

3. Why Do We Need Pointers?

1. Modify variables inside functions
 2. Dynamic memory allocation
 3. Arrays & strings
 4. Data structures (linked lists, trees)
 5. Efficient parameter passing
-

4. Pointers vs Pass by Value

Without Pointer (No change)

```
void change(int x) {  
    x = 100;  
}
```

```
int main() {  
    int a = 10;  
    change(a);  
    cout << a; // 10  
}
```

With Pointer (Change happens)

```
void change(int* x) {  
    *x = 100;  
}
```

```
int main() {  
    int a = 10;  
    change(&a);  
    cout << a; // 100  
}
```

Use pointers when a function must modify the original variable.

5. Pointers and Arrays

An array name is itself a pointer.

```
int arr[3] = {10, 20, 30};  
int* p = arr;
```

```
cout << *p << endl;    // 10  
cout << *(p + 1) << endl; // 20  
cout << *(p + 2) << endl; // 30
```

Traversing an Array Using Pointers

```
for (int i = 0; i < 3; i++) {  
    cout << *(arr + i) << " ";  
}
```

6. Dynamic Memory Allocation (new and delete)

Use pointers when **memory size is not known at compile time**.

```
int* p = new int; // allocate memory  
  
*p = 50;  
  
cout << *p << endl;
```

```
delete p;    // free memory
```

Dynamic Array

```
int n;
```

```
cin >> n;
```

```
int* arr = new int[n];
```

```
for (int i = 0; i < n; i++)
```

```
    arr[i] = i * 2;
```

```
delete[] arr;
```

Always delete dynamically allocated memory to avoid memory leaks.

7. Pointer to Pointer

```
int x = 10;
```

```
int* p = &x;
```

```
int** pp = &p;
```

```
cout << x << endl;  // 10
```

```
cout << *p << endl;  // 10
```

```
cout << **pp << endl; // 10
```

When used?

- 2D arrays
- Dynamic matrices
- Advanced data structures

8. Null and Dangling Pointers

Dangling Pointer (Dangerous)

```
int* p = new int(10);  
  
delete p;  
  
cout << *p; // undefined behavior
```

Safe Practice

```
delete p;  
  
p = nullptr;
```

Null Pointer

```
int* p = nullptr;
```

9. Pointers vs References

Pointer	Reference
Can be NULL	Cannot be NULL
Can change target	Cannot change target
Uses *	Uses normal variable
Explicit memory	Cleaner syntax

Reference Example

```
void change(int& x) {  
    x = 100;  
}
```

Use references when possible, pointers when necessary.

10. Real Use Case: Swap Using Pointers

```
void swap(int* a, int* b) {  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
}  
  
int main() {  
    int x = 5, y = 10;  
    swap(&x, &y);  
    cout << x << " " << y; // 10 5  
}
```

11. Common Mistakes

Uninitialized pointer
Forgetting delete
Dereferencing NULL
Using pointer after delete

```
int* p; // uninitialized  
  
*p = 10; // crash
```

12. When SHOULD You Use Pointers?

- ✓ Dynamic memory
- ✓ Modify variables in functions
- ✓ Implement data structures
- ✓ Work with arrays efficiently
- ✓ System-level programming